

Segundo Proyecto Integrador

Módulo de React - Single Page Application

Consigna para alumnos

React · TypeScript · Vite · Zustand · Git/GitHub · Deploy

Junio 2026

Documento de entrega académica

Segundo Proyecto Integrador - Módulo de React

Consigna de desarrollo para estudiantes

El objetivo de este proyecto es integrar los contenidos del módulo de React, construyendo una Single Page Application (SPA) funcional, navegable e interactiva con React 19, TypeScript y Vite. El proyecto vive en el cliente: sin API propia ni base de datos, los datos se manejan con estado global y persisten en localStorage.

El proyecto deberá resolver una necesidad concreta, tener una propuesta visual clara y demostrar organización técnica tanto en el código como en el uso de Git y GitHub.

Este trabajo forma parte del cierre del módulo de React y deberá poder utilizarse como una pieza de portfolio. Se desarrolla durante junio de 2026.

Resumen ejecutivo

Aspecto	Requisito
Modalidad	Grupos de 5 estudiantes (una idea por grupo)
Duración	Aproximadamente 1 mes (junio 2026)
Tecnologías	React 19 + TypeScript + Vite
Estado	Estado global (Zustand/Context), sin API ni base de datos
Persistencia	Uso obligatorio de localStorage (vía Zustand persist)
Estructura	Arquitectura feature-first, lógica pura en utils.ts
Reparto	5 desafíos de peso parejo (D1-D5): uno por integrante
GitHub	Repositorio público, una rama por feature, PRs chicos
Commits	Conventional Commits, historial claro por integrante
Entrega	Link al repositorio y link al deploy funcional

Indice

1. Modalidad de trabajo
2. Entrega final
3. Stack común y tecnologías
4. Ideas disponibles (20)
5. Git, GitHub y deploy

- 6. README
- 7. Calidad y accesibilidad
- 8. Criterios de aprobación
- 9. Observación final

1. Modalidad de trabajo

El proyecto se realizará en grupos de 5 estudiantes. La duración estimada de desarrollo será de aproximadamente un mes, durante junio de 2026.

Cada grupo desarrolla una sola idea. Cada idea trae 5 desafíos de peso parejo (D1 a D5): uno por integrante. El identificador marca dependencias, no dificultad.

D1 es siempre la base: la levanta un integrante primero y DESBLOQUEA al resto. Los demás desafíos cuelgan de ahí para trabajar en paralelo.

2. Entrega final

Cada grupo deberá entregar:

- Link al repositorio de GitHub.
- Link al deploy funcional de la SPA.

Dentro del repositorio deberán estar incluidos:

- Código completo del proyecto (React + TypeScript + Vite).
- Archivo README.md.
- Estructura de carpetas feature-first organizada.
- Historial de commits claro y progresivo.
- Pull Requests por feature integrados sobre la rama principal.

3. Stack común y tecnologías

Punto de partida para los 20 proyectos: **React 19 + Vite + TypeScript**. Ordenado pero sin sobre-ingeniería; es para un proyecto de 1 mes, no para escalar a 50 devs. Todas las versiones están verificadas a junio 2026.

Stack común (todos los proyectos)

Rol	Librería	Notas
Bundler / dev	Vite + @vitejs/plugin-react	dev server y build
UI	React 19 + TypeScript	base
Routing	React Router v7 (react-router-dom), modo librería (SPA)	alternativa con tipado fuerte: TanStack Router v1
Estado global	Zustand v5 + middleware persist	persiste en localStorage sin código disperso
Estilos	Tailwind CSS v4 vía @tailwindcss/vite	sin tailwind.config.js; tokens con @theme en el CSS
Iconos	lucide-react	

Rol	Librería	Notas
Clases condicionales	<code>clsx</code> + <code>tailwind-merge</code>	opcional
IDs	<code>crypto.randomUUID()</code> nativo	sin librería

Regla de estado: si el estado es de una sola pantalla, usá `useState/useReducer` + `Context`. No metas `Zustand` "porque sí" — reservalo para estado que cruza rutas o componentes lejanos.

Según necesidad (no en todos los proyectos)

Necesidad	Recomendado (moderno)	Evitar / nota
Formularios + validación	React Hook Form + Zod 4 + @hookform/resolvers	Zod es la fuente de verdad de tipos y reglas
Fechas	date-fns v4	tree-shakeable, TS-first; nada de Moment
Drag and drop	@dnd-kit	<code>react-beautiful-dnd</code> está discontinuado, no lo uses
Resaltado de código	Shiki (calidad VS Code)	highlight.js si querés algo liviano client-side
Animación	Motion v12 (<code>motion/react</code>)	ex <code>framer-motion</code> ; mismo API, nuevo paquete
Imprimir / PDF	CSS <code>@media print</code> primero, react-to-print si necesitás control	
Código QR	qrcode.react	
Color / contraste	fórmula WCAG a mano, o culori para conversiones	
Render de Markdown	react-markdown	
Gráficos	recharts	solo si la idea lo pide
Notificaciones / componentes	sonner (toasts), shadcn/ui sobre Radix (primitivas accesibles)	

Estructura de carpetas (feature-first)

```

src/
  app/
    router.tsx      # rutas (React Router v7)
    providers.tsx   # providers globales
    features/       # una carpeta por dominio
      <feature>/
        components/ # UI propia del feature
        hooks/      # lógica reutilizable del feature
        store.ts     # slice de Zustand (si aplica)
        schema.ts    # esquemas Zod (si aplica)
        utils.ts     # helpers puros: cálculos, formato
        types.ts
    shared/         # transversal a varios features
      components/ui/ # primitivas (Button, Input, Card)
      hooks/
      lib/          # helpers genéricos (storage, fechas)
    main.tsx
    index.css       # @import "tailwindcss";

```

Principios

- **Feature-first:** agrúpala por dominio (**wardrobe/**, **expenses/**), no por tipo técnico. El proyecto "grita" lo que hace.
- **Colocá lo que cambia junto:** componente, hook, store y schema del mismo feature, juntos.
- **shared/ solo cuando se reusa de verdad** — no es un cajón de sastre.
- **Lógica pura fuera de React:** cálculos (balances, contraste, fechas) en **utils.ts** como funciones puras, fáciles de razonar y reutilizar.
- **localStorage vía Zustand persist**, nunca **localStorage.setItem** disperso por la app.

Trabajo en equipo (grupos de 5)

Cada idea trae 5 desafíos de peso parejo: **uno por integrante**. El grafo de dependencias dice el orden de arranque.

- **Integrante 1 — Base (D1):** levanta el proyecto (Vite + router + store global), define el **modelo de datos compartido** y el layout/listado base. Arranca primero porque **desbloquea al resto**. Después coordina la integración.
- **Integrantes 2–5 (D2–D5):** cada uno toma un desafío que cuelga de la base y lo desarrolla en **su propia rama**, en paralelo.
- **Flujo Git:** una rama por feature, PRs chicos, el dueño de la base revisa e integra. Evitá que 5 personas toquen el mismo archivo.
- **Regla de oro:** si un desafío depende de otro que no sea D1, esos dos integrantes **acuerdan la interfaz (tipos/props) ANTES de empezar** y trabajan en paralelo contra ese contrato.

Calidad (opcional pero recomendado)

- **ESLint 9** (flat config) — o **Biome** como alternativa todo-en-uno (lint + format).

4. Ideas disponibles

Cada grupo elige UNA de las siguientes 20 ideas. La estrella original indica la carga global del proyecto; el rango es angosto a propósito para que ningún grupo trabaje mucho más que otro. Cada idea trae 5 desafíos (D1-D5) de peso parejo y un reparto sugerido entre los 5 integrantes.

Idea 1: Armario Virtual y Combinador de Looks

Dificultad: media (3/4)

App donde el usuario carga sus prendas y la herramienta arma combinaciones de ropa coherentes. Resuelve el clásico "tengo ropa pero no sé qué ponerme".

Conceptos que ejercita: estado global para el armario, derivación de datos (combinaciones a partir de prendas), filtros, manejo de imágenes vía URL/base64.

Librerías recomendadas: solo el stack base. Estado del armario con **Zustand + persist**; imágenes por URL o base64, sin librería.

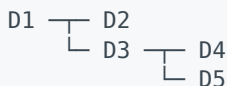
Recomendaciones de diseño/implementación

- Modelá la prenda como objeto con **id**, **category** (superior, inferior, calzado, abrigo), **color**, **season**, **occasion**, **image**, **available**.
- Una combinación es una prenda de cada categoría necesaria; armala filtrando por compatibilidad, no guardando todas las combinaciones posibles.
- Vista tipo "armario visual" en grilla.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + modelo de prenda + carga (imagen, categoría, color, temporada, ocasión) + grilla tipo armario. · *base*
- **D2** — Filtro combinado por varios atributos a la vez (color + temporada + ocasión). · *depende de D1*
- **D3** — Motor de combinaciones: arma un look tomando una prenda por categoría sin repetir incompatibles. · *depende de D1*
- **D4** — Favoritos: guardar y quitar combinaciones, persistidas en **localStorage**. · *depende de D3*
- **D5** — Generador aleatorio bajo consigna ("formal", "verano", "lluvia") respetando disponibilidad ("en lavado"). · *depende de D3*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, modelo de prenda, carga y grilla.
- **I2 (D2):** sistema de filtros combinados.
- **I3 (D3):** motor de combinaciones (acuerda con I4/I5 el shape del "look").
- **I4 (D4):** favoritos + persistencia.
- **I5 (D5):** generador aleatorio por consigna.

Idea 2: Planificador Visual de Escritorio

Dificultad: alta (4/4)

Herramienta para diseñar la distribución de un escritorio de estudio, trabajo o gaming, ubicando objetos en un espacio y avisando si está sobrecargado.

Conceptos que ejercita: posicionamiento en pantalla, estado de una colección de elementos con coordenadas, cálculo derivado (ocupación), drag para mover objetos.

Librerías recomendadas: `@dnd-kit` (mover y redimensionar objetos), `Zustand + persist` (guardar configuraciones).

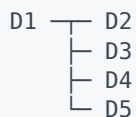
Recomendaciones de diseño/implementación

- Cada objeto: `id`, `type` (monitor, teclado, planta...), `width`, `height`, `x`, `y`. El escritorio es un contenedor con dimensiones fijas.
- "Sobrecargado" = suma de áreas de objetos / área del escritorio supera un umbral.

Desafíos (todos ★★★★★)

- **D1** — Base: scaffold + modelo de objeto + render posicionado dentro del área del escritorio + alta de objetos. · *base*
- **D2** — Mover los objetos con drag y persistir su posición en el estado. · *depende de D1*
- **D3** — Calcular el porcentaje de ocupación y alertar cuando supera el umbral. · *depende de D1*
- **D4** — Guardar y cargar varias configuraciones de escritorio. · *depende de D1*
- **D5** — Plantillas que precargan objetos (estudio, gaming, minimalista) + redimensionar objetos en el área. · *depende de D1*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, modelo de objeto, área del escritorio, alta y render.
- **I2 (D2):** drag y persistencia de posición.
- **I3 (D3):** cálculo de ocupación y alertas.
- **I4 (D4):** guardar/cargar configuraciones.
- **I5 (D5):** plantillas + redimensionado.

Idea 3: Seguidor de Postulaciones Laborales

Dificultad: media (3/4)

Panel para llevar el control de a qué trabajos te postulaste, en qué estado están y qué falta hacer. Evita perder el hilo cuando hay muchas búsquedas abiertas.

Conceptos que ejercita: CRUD con estado global, máquina de estados simple, filtros, fechas, datos derivados para el resumen.

Librerías recomendadas: `React Hook Form + Zod` (alta/edición), `date-fns` (días sin seguimiento), `Zustand + persist`, `recharts` opcional (mini-gráfico del resumen).

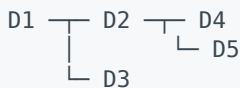
Recomendaciones de diseño/implementación

- Postulación: **company, position, date, status** (postulado, entrevista, oferta, rechazado), **priority, notes, link**.
- Pensá los estados como un flujo; el "tiempo sin seguimiento" se calcula contra la fecha del último cambio.
- El panel resumen son datos derivados del array, no estado aparte.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de postulaciones (todos los campos) + listado y vista de detalle por router. · *base*
- **D2** — Flujo de estados (máquina simple) con cambio de estado y reflejo visual. · *depende de D1*
- **D3** — Filtros combinados por estado, empresa, fecha o prioridad. · *depende de D1*
- **D4** — Alertas por días sin movimiento (cálculo contra el último cambio). · *depende de D2*
- **D5** — Panel resumen con conteos derivados + mini-gráfico (activas, entrevistas, rechazos). · *depende de D2*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, CRUD, listado y detalle ruteado.
- **I2 (D2):** máquina de estados + UI de cambio (acuerda con I4/I5 el campo de fecha de último cambio).
- **I3 (D3):** filtros combinados.
- **I4 (D4):** alertas por inactividad.
- **I5 (D5):** panel resumen + gráfico.

Idea 4: Registro de Garantías con Vencimientos

Dificultad: media (3/4)

App para guardar las garantías de tus productos y saber cuáles siguen vigentes, cuáles están por vencer y cuáles ya caducaron.

Conceptos que ejercita: cálculo de fechas, clasificación por estado derivado, filtros, carga de archivos/imágenes.

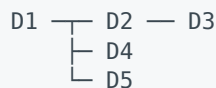
Librerías recomendadas: React Hook Form + Zod, date-fns (vencimientos), input **file** nativo para el comprobante, Zustand + persist.

Recomendaciones de diseño/implementación

- Producto: **name, category, purchaseDate, durationMonths, receipt**. La fecha de vencimiento se **calcula**, no se guarda.
- Definí "próxima a vencer" con una ventana clara (ej. faltan ≤ 30 días).
- Diferenciá los tres estados con color/etiqueta, no solo con texto.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de productos (fecha de compra y duración) + listado. · *base*
- **D2** — Cálculo de la fecha de vencimiento de cada garantía (lógica pura en `utils.ts`). · *depende de D1*
- **D3** — Clasificación visual: vigente, próxima a vencer, vencida (con color/etiqueta). · *depende de D2*
- **D4** — Búsqueda y filtros por categoría, fecha o nombre. · *depende de D1*
- **D5** — Adjuntar imagen del comprobante + recordatorios de las próximas a vencer. · *depende de D1*

Dependencias**Reparto sugerido (5 integrantes)**

- **I1 · Base (D1):** scaffold, CRUD, modelo, listado.
- **I2 (D2):** cálculo de vencimientos (acuerda con I3 el campo de estado derivado).
- **I3 (D3):** clasificación visual por estado.
- **I4 (D4):** búsqueda y filtros.
- **I5 (D5):** adjunto de comprobante + recordatorios.

Idea 5: Generador de Reportes de Bugs

Dificultad: media (3/4)

Formulario que transforma la descripción de un bug en un reporte limpio y copiable en Markdown, listo para pegar en un issue.

Conceptos que ejercita: formularios controlados, validación, generación de texto a partir de estado, vista previa en tiempo real.

Librerías recomendadas: **React Hook Form + Zod**, **react-markdown** (vista previa del Markdown), **navigator.clipboard** nativo para copiar.

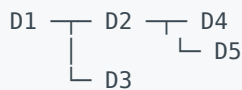
Recomendaciones de diseño/implementación

- Campos: `title`, `description`, `steps`, `expectedResult`, `actualResult`, `severity`, `priority`, `environment`, `tone`.
- El Markdown es una función pura de los campos; mostralo en vivo en una vista previa.
- "Tono" cambia el wording de las secciones (formal, directo, detallado).

Desafíos (todos ★★★)

- **D1** — Base: scaffold + formulario controlado con todos los campos. · *base*
- **D2** — Generación del reporte en Markdown a partir de los campos (función pura). · *depende de D1*
- **D3** — Validación de campos mínimos antes de generar (Zod). · *depende de D1*
- **D4** — Vista previa en vivo (**react-markdown**) + copiar al portapapeles. · *depende de D2*
- **D5** — Plantillas por tipo de bug (visual, funcional, rendimiento) + selector de tono. · *depende de D2*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold + formulario controlado.
- **I2 (D2):** motor de generación de Markdown (acuerda el formato con I4/I5).
- **I3 (D3):** validación con Zod.
- **I4 (D4):** vista previa en vivo + copiar.
- **I5 (D5):** plantillas + tonos.

Idea 6: Tarjetas de Estudio con Repaso Inteligente

Dificultad: media (3/4)

Flashcards con modos de repaso y quiz que priorizan automáticamente lo que más te cuesta.

Conceptos que ejercita: estado global del mazo, lógica de priorización, registro de métricas por tarjeta, navegación entre modos con router.

Librerías recomendadas: Zustand + persist, date-fns (rachas diarias), Motion opcional (animación de flip de la tarjeta).

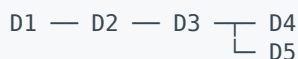
Recomendaciones de diseño/implementación

- Tarjeta: **question**, **answer**, **topic**, **difficulty**, **hits**, **misses**.
- La priorización es un orden derivado (más errores → aparece antes); una fórmula simple alcanza.
- Separá "modo repaso" (ves la respuesta) de "modo quiz" (la adiviná y se registra).

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de tarjetas (pregunta, respuesta, tema, dificultad) + store global. · *base*
- **D2** — Modo repaso (mostrar/ocultar respuesta) y modo quiz, separados por router. · *depende de D1*
- **D3** — Registro de aciertos y errores por tarjeta. · *depende de D2*
- **D4** — Priorización de las tarjetas más falladas al armar la sesión. · *depende de D3*
- **D5** — Progreso por tema/sesión + rachas diarias. · *depende de D3*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, CRUD de tarjetas, store.
- **I2 (D2):** modos repaso/quiz ruteados (acuerda con I3 cómo se registran resultados).
- **I3 (D3):** registro de métricas por tarjeta.
- **I4 (D4):** algoritmo de priorización.
- **I5 (D5):** progreso + rachas.

Idea 7: Chequeo Rápido de Accesibilidad Visual

Dificultad: media (3/4)

Herramienta educativa que revisa contraste de colores y otros puntos básicos de accesibilidad, devolviendo recomendaciones claras.

Conceptos que ejercita: cálculo (ratio de contraste), validación de reglas, feedback visual por estado, manejo de inputs de color.

Librerías recomendadas: la fórmula WCAG (luminancia relativa) a mano alcanza; **culori** opcional para conversiones de color. Sin más que el stack base.

Recomendaciones de diseño/implementación

- El contraste se calcula con la fórmula WCAG; buscá la fórmula, no la inventes.
- Usá semáforo visual: correcto / riesgoso / incorrecto, con el ratio numérico al lado.
- Mantené el alcance acotado: contraste + alt text + estructura de headings ya es suficiente.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + layout con secciones de chequeo + inputs de color y preview de la combinación. · *base*
- **D2** — Cálculo del ratio de contraste (fórmula WCAG) e indicación de si pasa AA/AAA. · *depende de D1*
- **D3** — Validación de texto alternativo en un componente/imagen de ejemplo. · *independiente*
- **D4** — Revisión de la estructura de headings (que no salten niveles). · *independiente*
- **D5** — Sugerencia de combinaciones de colores accesibles cercanas a la elegida. · *depende de D2*

Dependencias

D1 — D2 — D5
D3 (independiente)
D4 (independiente)

Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, layout de chequeos, inputs de color.
- **I2 (D2):** cálculo de contraste WCAG.
- **I3 (D3):** chequeo de alt text (frente independiente).
- **I4 (D4):** chequeo de headings (frente independiente).
- **I5 (D5):** sugeridor de colores accesibles.

Idea 8: Armador de Listas para Viajar

Dificultad: media (3/4)

Generador de listas para preparar mochila, bolso o valija según contexto, duración y clima, con plantillas reutilizables.

Conceptos que ejercita: generación de listas a partir de reglas, estado de checklist, plantillas, datos semilla en JSON.

Librerías recomendadas: solo el stack base — JSON mock importado + **Zustand** + **persist**.

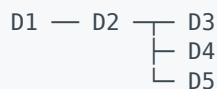
Recomendaciones de diseño/implementación

- Tené un JSON mock con elementos sugeridos por contexto (universidad, viaje, lluvia, mudanza).
- La lista generada se puede editar; separá "sugerido" de "agregado por el usuario".
- El clima opcional suma/quita ítems (paraguas, abrigo) sin rehacer la lista.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + selección de contexto/duración/tipo + estructura de la lista + store. · *base*
- **D2** — Generación automática de la lista desde el JSON mock según la selección. · *depende de D1*
- **D3** — Edición manual: agregar, editar y eliminar elementos (separando sugerido de propio). · *depende de D2*
- **D4** — Checklist persistido: marcar pendientes / guardados. · *depende de D2*
- **D5** — Perfiles/plantillas reutilizables + ajuste de la lista según clima. · *depende de D2*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, selección de contexto, estructura de lista, store.
- **I2 (D2):** generador desde el mock (acuerda con el resto el shape del ítem).
- **I3 (D3):** edición manual de ítems.
- **I4 (D4):** checklist persistido.
- **I5 (D5):** perfiles/plantillas + ajuste por clima.

Idea 9: Divisor de Gastos Compartidos

Dificultad: alta (4/4)

App para registrar gastos de un grupo (viaje, depto compartido, salida) y calcular automáticamente quién le debe cuánto a quién. *(Reemplaza al inventario genérico original: misma familia de skills, pero con lógica de cálculo real y más motivadora.)*

Conceptos que ejercita: estado global del grupo y gastos, cálculos derivados (balances), algoritmo simple de saldos, presentación clara de resultados.

Librerías recomendadas: stack base; toda la matemática (balances, liquidación) en `utils.ts` como funciones puras. **React Hook Form + Zod** para el alta de gastos.

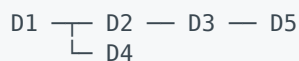
Recomendaciones de diseño/implementación

- Modelo: `participants[]`, y `expenses[]` con `description`, `amount`, `paidBy`, `participants` (entre quiénes se divide).
- El balance de cada persona = lo que pagó – lo que le corresponde. La liquidación ("quién paga a quién") es lo más fino; cuidá el redondeo.
- Mostrá los números siempre visibles para que el cálculo sea transparente.

Nota de equipo: esta es la idea más secuencial (balance necesita el split, liquidación necesita el balance). I2 → I3 → I5 deben acordar interfaces temprano y trabajar contra tipos mockeados en paralelo.

Desafíos (todos ★★★★★)

- **D1** — Base: scaffold + grupo con participantes + alta de gastos (quién pagó, monto, descripción). · *base*
- **D2** — División de cada gasto entre participantes elegidos en partes iguales. · *depende de D1*
- **D3** — Cálculo del balance de cada persona (a favor / en contra). · *depende de D2*
- **D4** — Filtros y vistas de gastos por persona/categoría + historial. · *depende de D1*
- **D5** — Liquidación mínima: los pagos concretos para saldar todas las deudas. · *depende de D3*

Dependencias**Reparto sugerido (5 integrantes)**

- **I1 · Base (D1):** scaffold, modelo, grupo y alta de gastos.
- **I2 (D2):** lógica de división (define el tipo `Split` que consume I3).
- **I3 (D3):** cálculo de balances (define el tipo `Balance` que consume I5).
- **I4 (D4):** filtros, vistas e historial (frente paralelo, solo depende de la base).
- **I5 (D5):** algoritmo de liquidación mínima.

Idea 10: Organizador de Reuniones y Próximos Pasos

Dificultad: media (3/4)

Herramienta para cargar notas de una reunión y obtener un resumen ordenado con decisiones, acuerdos y acciones pendientes con responsables.

Conceptos que ejercita: procesamiento de texto, estructuras de datos para acciones, asignación responsable/fecha, generación de resumen.

Librerías recomendadas: `date-fns` (fechas de acciones), `navigator.clipboard` para copiar el resumen, `Zustand + persist`.

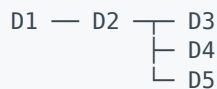
Recomendaciones de diseño/implementación

- Empezá con secciones manuales (decisiones / acuerdos / pendientes) antes de intentar "extraer" del texto.
- La extracción de acciones puede ser simple: detectar líneas con verbos o un prefijo tipo `[ ]`.
- El resumen final es texto generado a partir del estado; agregá botón de copiar.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + carga de notas + separación en decisiones, acuerdos y pendientes. · *base*
- **D2** — Acciones pendientes como ítems estructurados (modelo + UI). · *depende de D1*
- **D3** — Extracción de acciones candidatas desde el texto ingresado (regla simple). · *depende de D2*
- **D4** — Asignación de responsable y fecha a cada acción. · *depende de D2*
- **D5** — Resumen final ordenado y copiable + plantilla inicial de carga. · *depende de D2*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, carga de notas, secciones.
- **I2 (D2):** modelo y UI de acciones (define el tipo **Action** que usan I3/I4/I5).
- **I3 (D3):** extractor de acciones del texto.
- **I4 (D4):** asignación de responsable/fecha.
- **I5 (D5):** generador de resumen + plantilla.

Idea 11: Generador de Contraseñas Memorables

Dificultad: media (3/4)

Crea contraseñas fáciles de recordar combinando frases raras (objeto + acción + lugar + número) y estima su nivel de seguridad.

Conceptos que ejercita: generación aleatoria controlada, cálculo de un puntaje (entropía), manejo del portapapeles, listas semilla.

Librerías recomendadas: stack base; listas en JSON; **navigator.clipboard** nativo. **No persistas** contraseñas: esto genera, no almacena credenciales.

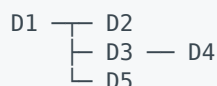
Recomendaciones de diseño/implementación

- Mantené listas (objetos, acciones, lugares) en un JSON y combinalas al azar.
- El "nivel de seguridad" es una estimación simple por entropía (longitud, variedad de caracteres, números); explicá que es orientativo.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + generación de una frase combinando objeto, acción, lugar y número desde las listas. · *base*
- **D2** — Personalización: longitud, cantidad de palabras, separadores, incluir números/símbolos. · *depende de D1*
- **D3** — Estimación de seguridad por entropía + medidor visual. · *depende de D1*
- **D4** — Copiar al portapapeles + recomendaciones accionables para mejorar la contraseña. · *depende de D3*
- **D5** — Generar un lote de opciones + historial de sesión (en memoria, NO persistido) + advertencias de reutilización. · *depende de D1*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, listas JSON, generador base.
- **I2 (D2):** opciones de personalización.

- **I3 (D3):** cálculo de entropía + medidor.
- **I4 (D4):** copiar + recomendaciones.
- **I5 (D5):** lote de opciones + historial en memoria.

Idea 12: Entrenador de Atajos de Teclado

Dificultad: media (3/4)

Juego para practicar atajos de teclado de distintas herramientas, detectando las combinaciones reales que presiona el usuario.

Conceptos que ejercita: manejo de eventos de teclado, detección de combinaciones, registro de tiempo/aciertos, niveles.

Librerías recomendadas: eventos `keydown` nativos (`react-hotkeys-hook` opcional para simplificar), `date-fns` para tiempos, `Zustand` + `persist` (récores).

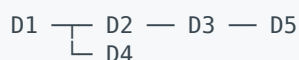
Recomendaciones de diseño/implementación

- Capturá `keydown` y reconstruí la combinación (`Ctrl+Shift+P`). Cuidado con `preventDefault` para no disparar atajos del navegador.
- Cada desafío: `description`, `expectedCombo`, `tool`, `level`.
- Feedback inmediato: verde si acertó, rojo + la combinación correcta si falló.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + catálogo de desafíos (JSON) + mostrar la combinación objetivo. · *base*
- **D2** — Detección de la combinación real (`keydown`) y comparación con la esperada. · *depende de D1*
- **D3** — Registro de errores, aciertos y tiempo de respuesta. · *depende de D2*
- **D4** — Organización de atajos por herramienta (navegador, editor, diseño) y nivel. · *depende de D1*
- **D5** — Récord personal, progreso general y feedback inmediato al equivocarse. · *depende de D3*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, catálogo JSON, render del desafío.
- **I2 (D2):** detector de combinaciones (define el evento que consume I3).
- **I3 (D3):** registro de métricas y tiempos.
- **I4 (D4):** organización por herramienta/nivel (frente paralelo).
- **I5 (D5):** récord, progreso, feedback.

Idea 13: Decisor con Criterios Ponderados

Dificultad: media (3/4)

App para tomar decisiones difíciles comparando opciones contra criterios con distinto peso, y mostrando un ganador justificado.

Conceptos que ejercita: matrices de datos, cálculo ponderado, recálculo reactivo, presentación tabular.

Librerías recomendadas: solo el stack base; la lógica ponderada vive en `utils.ts` como funciones puras.

Recomendaciones de diseño/implementación

- Modelo: `options[]`, `criteria[]` (con `weight`), y una matriz `scores[option][criterion]`.
- Puntaje final = Σ (puntaje $\times$ peso). Normalizá los pesos para que sumen 1 o 100.
- La tabla comparativa es clave: que el usuario vea de dónde sale el resultado.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + carga de opciones + estructura de la matriz + store. · *base*
- **D2** — Definición de criterios y asignación de pesos (normalizados). · *depende de D1*
- **D3** — Puntuación de cada opción según cada criterio (matriz opción $\times$ criterio). · *depende de D2*
- **D4** — Cálculo del ganador + tabla comparativa transparente. · *depende de D3*
- **D5** — Sliders de peso en vivo que recalculan el ranking al instante. · *depende de D4*

Dependencias

D1 — D2 — D3 — D4 — D5

Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, opciones, estructura de matriz, store.
- **I2 (D2):** criterios y pesos (define el tipo `Criterion`).
- **I3 (D3):** UI de puntuación de la matriz.
- **I4 (D4):** cálculo del ganador + tabla.
- **I5 (D5):** sliders reactivos.

Nota de equipo: idea bastante secuencial (matriz $\rightarrow$ puntaje $\rightarrow$ ganador $\rightarrow$ sliders). Acuerden los tipos `Option`, `Criterion` y `scores` en el día 1 para trabajar en paralelo contra mocks.

Idea 14: Tablero de Lecturas y Recursos

Dificultad: media (3/4)

Tablero tipo Kanban para organizar artículos, videos y lecturas educativas por estado de avance.

Conceptos que ejercita: estado por columnas, filtros, notas, drag entre columnas, datos derivados (contadores).

Librerías recomendadas: `@dnd-kit` (mover entre columnas), `React Hook Form + Zod` (alta de recursos), `Zustand + persist`.

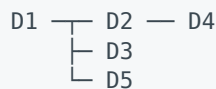
Recomendaciones de diseño/implementación

- Recurso: `title`, `link`, `topic`, `priority`, `difficulty`, `status` (pendiente, en progreso, terminado), `notes`.
- El contador de pendientes son datos derivados, no estado aparte.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de recursos + store global. · *base*
- **D2** — Tablero con columnas por estado. · *depende de D1*
- **D3** — Filtros por tema, prioridad, dificultad y estado. · *depende de D1*
- **D4** — Mover recursos entre estados con drag-and-drop ([@dnd-kit](#)). · *depende de D2*
- **D5** — Notas por recurso + sección de favoritos + contador de pendientes. · *depende de D1*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, CRUD, store.
- **I2 (D2):** tablero por columnas (define el contrato que usa I4).
- **I3 (D3):** filtros.
- **I4 (D4):** drag-and-drop entre columnas.
- **I5 (D5):** notas + favoritos + contadores.

Idea 15: Agenda de Mantenimiento del Hogar

Dificultad: media (3/4)

Registro de tareas recurrentes del hogar que calcula cuándo toca hacer cada una y avisa de las vencidas.

Conceptos que ejercita: fechas y recurrencia, clasificación por urgencia derivada, filtros, notas/costos.

Librerías recomendadas: `date-fns` (recurrencia y próxima fecha), **React Hook Form + Zod, Zustand + persist**.

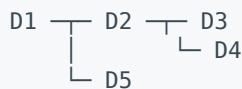
Recomendaciones de diseño/implementación

- Tarea: `name`, `area`, `frequencyDays`, `lastDoneDate`, `notes`, `cost`. La próxima fecha = última + frecuencia.
- A diferencia de las garantías (vencimiento único), acá la recurrencia es el corazón: al marcar "hecha", se recalcula la próxima.
- Clasificá por urgencia: vencida, próxima, al día.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de tareas (zona, frecuencia, última fecha) + listado. · *base*
- **D2** — Cálculo de la próxima fecha recomendada (lógica pura). · *depende de D1*
- **D3** — Clasificación por urgencia/estado (vencida, próxima, al día). · *depende de D2*
- **D4** — Marcar tarea como hecha → recalcular su próxima fecha. · *depende de D2*
- **D5** — Categorías (limpieza, reparaciones, revisiones) + filtros + notas/costos. · *depende de D1*

Dependencias

**Reparto sugerido (5 integrantes)**

- **I1 · Base (D1):** scaffold, CRUD, modelo, listado.
- **I2 (D2):** cálculo de próxima fecha (define el campo derivado que usan I3/I4).
- **I3 (D3):** clasificación por urgencia.
- **I4 (D4):** marcar hecha + recálculo.
- **I5 (D5):** categorías, filtros, notas/costos (frente paralelo).

Idea 16: Biblioteca de Snippets de Código

Dificultad: media (3/4)

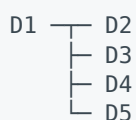
Repositorio personal de fragmentos de código reutilizables, con búsqueda, etiquetas y copia rápida.

Conceptos que ejercita: CRUD con estado global, búsqueda de texto, etiquetas, integración de resaltado de sintaxis.**Librerías recomendadas:** **Shiki** (resaltado calidad VS Code) o **highlight.js** si querés algo más liviano client-side; **navigator.clipboard**; **Zustand + persist**.**Recomendaciones de diseño/implementación**

- Snippet: **title**, **language**, **code**, **description**, **tags[]**, **favorite**.
- La búsqueda debe mirar dentro del código y la descripción, no solo el título.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de snippets (todos los campos) + store global. · *base*
- **D2** — Filtros por lenguaje, etiqueta y favoritos. · *depende de D1*
- **D3** — Copiar el snippet al portapapeles con un clic. · *depende de D1*
- **D4** — Búsqueda por palabra clave dentro del código o la descripción. · *depende de D1*
- **D5** — Resaltado de sintaxis según lenguaje (Shiki / highlight.js). · *depende de D1*

Dependencias**Reparto sugerido (5 integrantes)**

- **I1 · Base (D1):** scaffold, CRUD, store.
- **I2 (D2):** filtros.
- **I3 (D3):** copiar al portapapeles.
- **I4 (D4):** búsqueda en código/descripción.
- **I5 (D5):** integración del resaltado de sintaxis.

Nota: idea con paralelismo ideal — los 4 frentes cuelgan directo de la base.

Idea 17: Laboratorio de Validación de Formularios

Dificultad: alta (4/4)

Herramienta para armar formularios con reglas de validación combinables y ver cómo responden en tiempo real. Ideal para entender validación a fondo.

Conceptos que ejercita: formularios dinámicos, motor de reglas, validación en tiempo real, estado por campo.

Librerías recomendadas: **Zod 4** como motor de reglas + **React Hook Form** (con [@hookform/resolvers](#)) para el estado y la validación en tiempo real. Es el núcleo del proyecto.

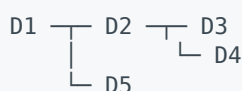
Recomendaciones de diseño/implementación

- Campo configurable: `label`, `type`, y `rules[]` (requerido, min, max, email, regex) con su mensaje.
- Pensá la validación como funciones puras `(value, rule) => error | null`; así combinás reglas fácil.
- Mostrá el estado por campo: válido, inválido, pendiente.

Desafíos (todos ★★★★★)

- **D1** — Base: scaffold + creación dinámica de campos + store del formulario. · *base*
- **D2** — Motor de reglas combinables por campo (requerido, min, max, email, regex). · *depende de D1*
- **D3** — Mensajes de error personalizados según la regla que falla. · *depende de D2*
- **D4** — Validación en tiempo real + estado por campo + resumen de errores activos. · *depende de D2*
- **D5** — Galería de formularios prearmados (login, signup, checkout) usando el motor. · *depende de D1*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, creación dinámica de campos, store.
- **I2 (D2):** motor de reglas (define el tipo `Rule` que consumen I3/I4).
- **I3 (D3):** mensajes de error personalizados.
- **I4 (D4):** validación en tiempo real + resumen.
- **I5 (D5):** galería de formularios de ejemplo (frente paralelo).

Idea 18: Cápsulas de Mensajes a Futuro

Dificultad: media (3/4)

Escribís un mensaje y elegís una fecha; queda bloqueado hasta que llegue ese día, con cuenta regresiva.

Conceptos que ejercita: fechas y temporizadores, estado bloqueado/desbloqueado derivado, validación, persistencia.

Librerías recomendadas: `date-fns` (comparación de fechas), `setInterval` nativo para la cuenta regresiva, `Motion` opcional (transición al abrir), `Zustand + persist`.

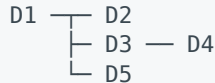
Recomendaciones de diseño/implementación

- Cápsula: `message`, `openDate`, `category`. El estado (bloqueada/abierta/próxima) se deriva de comparar con "ahora".
- La cuenta regresiva necesita un `setInterval` dentro de un `useEffect` con su limpieza correspondiente.
- Sin DB, la "magia" depende de que el dato sobreviva al recargar (`persist`).

Desafíos (todos ★★★)

- **D1** — Base: scaffold + CRUD de cápsulas (mensaje, fecha) + store con `persist`. · *base*
- **D2** — Validación de que la fecha sea futura antes de guardar (Zod/RHF). · *depende de D1*
- **D3** — Bloqueo del contenido hasta que llegue la fecha (estado derivado). · *depende de D1*
- **D4** — Cuenta regresiva en vivo (`setInterval`) + diferenciar abiertas / bloqueadas / próximas. · *depende de D3*
- **D5** — Categorías (metas, recuerdos, promesas) + vista especial para cápsulas abiertas. · *depende de D1*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, CRUD, store con `persist`.
- **I2 (D2):** validación de fecha futura.
- **I3 (D3):** lógica de bloqueo derivado (define el estado que consume I4).
- **I4 (D4):** cuenta regresiva en vivo.
- **I5 (D5):** categorías + vista de abiertas (frente paralelo).

Idea 19: Lector Asistido de Contratos

Dificultad: media (3/4)

Pegás un texto largo (contrato, términos) y la herramienta resalta términos clave y arma un resumen de los puntos que merecen atención.

Conceptos que ejercita: procesamiento de texto, resaltado, clasificación por palabras clave, búsqueda dentro del texto.

Librerías recomendadas: solo el stack base; el procesamiento de texto se hace a mano (`regex` / `split`). Opcional `Zustand + persist` si querés guardar análisis previos.

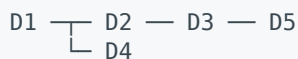
Recomendaciones de diseño/implementación

- Definí listas de palabras clave por categoría (plazos, multas, renovación, penalidades) y buscalas en el texto.
- Para resaltar, partí el texto y envolví las coincidencias en un `<mark>` con color según categoría.
- Dejá claro que **no reemplaza asesoramiento legal**; es una ayuda de lectura.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + área de análisis + carga de texto + catálogo de palabras clave (JSON). · *base*
- **D2** — Resaltado de palabras clave en el texto (`<mark>` por categoría). · *depende de D1*
- **D3** — Clasificación de fragmentos detectados por nivel de atención/categoría. · *depende de D2*
- **D4** — Búsqueda de palabras específicas dentro del texto. · *depende de D1*
- **D5** — Resumen de términos detectados, agrupados por categoría. · *depende de D3*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, área de texto, catálogo de palabras clave.
- **I2 (D2):** motor de resaltado (define el tipo `Match` que usan I3/I5).
- **I3 (D3):** clasificación de fragmentos.
- **I4 (D4):** búsqueda dentro del texto (frente paralelo).
- **I5 (D5):** resumen agrupado por categoría.

Idea 20: Tarjeta de Información de Emergencia

Dificultad: media (3/4)

Generador de una tarjeta imprimible con datos personales, contactos y datos médicos esenciales para llevar en la billetera o el celular.

Conceptos que ejercita: formularios, validación de campos críticos, vista previa, estilos para impresión, generación de QR.

Librerías recomendadas: `React Hook Form` + `Zod`, `react-to-print` (o CSS `@media print`), `qrcode.react` (QR con los datos), `Zustand` + `persist`.

Recomendaciones de diseño/implementación

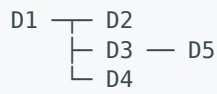
- Pedí solo lo necesario: nombre, contacto de emergencia, grupo sanguíneo, alergias, medicación. Evitá datos sensibles de más.
- Validá los campos críticos antes de habilitar la impresión/descarga.

Desafíos (todos ★★★)

- **D1** — Base: scaffold + formulario con datos personales, contactos y datos médicos + store. · *base*
- **D2** — Validación de campos críticos (nombre, contacto de emergencia, grupo sanguíneo). · *depende de D1*
- **D3** — Vista imprimible de la tarjeta (`@media print` / `react-to-print`). · *depende de D1*
- **D4** — Vista previa en vivo antes de imprimir/descargar. · *depende de D1*

- **D5** — Estilos de diseño elegibles + QR con los datos + aviso de actualización. · *depende de D3*

Dependencias



Reparto sugerido (5 integrantes)

- **I1 · Base (D1):** scaffold, formulario, store.
- **I2 (D2):** validación de campos críticos.
- **I3 (D3):** vista imprimible (define el layout que estiliza I5).
- **I4 (D4):** vista previa en vivo (frente paralelo).
- **I5 (D5):** estilos elegibles + QR + aviso.

5. Git, GitHub y deploy

El uso de Git y GitHub forma parte del proyecto. Cada grupo deberá cumplir con:

- Repositorio público en GitHub.
- Una rama por feature; el dueño de la base (D1) revisa e integra.
- Pull Requests chicos y frecuentes; evitar que 5 personas toquen el mismo archivo.
- Conventional Commits, claros y descriptivos.
- README completo y deploy funcional de la SPA.

Conventional Commits

```
feat: add wardrobe item form
feat: render looks from selected garments
fix: correct localStorage persist key
refactor: extract balance calc to utils
docs: update README with deploy link
```

Eviten commits genéricos como: cambios, avance, arreglos, commit final, cosas o asdf.

Deploy

El proyecto deberá estar publicado mediante un deploy funcional. Pueden usar Vercel, Netlify o GitHub Pages (con la configuración de SPA correspondiente para que el router funcione al recargar). El deploy debe permitir navegar correctamente toda la aplicación.

6. README

El repositorio deberá incluir un README.md con: nombre del proyecto, descripción breve, integrantes, idea elegida, tecnologías, funcionalidades principales, link al deploy, link al repositorio, instrucciones de uso y estructura del proyecto.

7. Calidad y accesibilidad

Recomendado pero no obligatorio: ESLint 9 (flat config) o Biome como alternativa todo-en-uno (lint + format). Se valora la accesibilidad básica:

- Imágenes con alt cuando corresponda.
- Contraste legible entre texto y fondo.
- Formularios con labels asociadas.
- Botones y enlaces identificables; navegación clara.
- No depender solo del color para transmitir información.
- Estados de error o éxito visibles en pantalla.

8. Criterios de aprobación

Para que el proyecto sea aprobado, deberá cumplir con los requisitos mínimos técnicos, de proceso y de entrega:

- SPA funcional en React 19 + TypeScript + Vite.
- Ruteo entre vistas con React Router (o equivalente).
- Estado global con Zustand/Context donde corresponda.
- Persistencia real en localStorage con sentido dentro del proyecto.
- Arquitectura feature-first y lógica pura en utils.ts.
- Los 5 desafíos (D1-D5) resueltos e integrados.
- Repositorio público con trabajo colaborativo en GitHub.
- Deploy funcional y README completo.
- Respetar la idea elegida y sus 5 desafíos.

9. Observación final

Este proyecto no busca solamente que la aplicación se vea bien. Debe demostrar que el grupo comprende cómo construir una SPA completa, organizada e interactiva: derivación de datos, estado global, persistencia, ruteo y composición de componentes.

Se evaluará la coherencia entre la idea elegida, la arquitectura, el estado, las funcionalidades, el proceso de trabajo en GitHub, la documentación y el deploy final. El objetivo es que cada grupo entregue un proyecto funcional, claro, útil y presentable como pieza de portfolio.